1. Implement copy command (cp) to copy a file content to other file using file management system calls

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
   char buf[]=" ";



   // Open first file
   int fd1 = open("C:\\Users\\staff\\Music\\Fwd OS Notes\\lab\\p1.txt", O_RDONLY);
   int fd2 = open("C:\\Users\\staff\\Music\\Fwd OS Notes\\lab\\p2.txt", O_WRONLY);
   if (fd1 == -1) {
      printf("Error opening first file\n");
      return 1; // Exit the program if opening fd1 fails
   }



   // Copy content from fd1 to fd2, one byte at a time
   while (read(fd1, buf, 1)) {
      write(fd2, buf, 1);
   }

   printf("Successful copy\n");
   // Close both files
   close(fd1);
   close(fd2);

   return 0; // Successful execution
}
```

2. Create four different processes (with different process ID) and assign four different tasks (addition, subtraction, Multiplication, division) to each process. All processes should display the result along with its process ID and parent process ID.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>  // Include this header for waitpid()

int main(int argc, char** argv) {
   int a, b, c, num1 = 20, num2 = 10;
   int wstatus;

   a = fork();
   if (a == 0) {
      printf("\nParent PPID : %d\n", getppid());
      printf("\nParent child ID is : %d\t Parent is :%d\n", getpid(), getppid());
      printf(" Addition is %d\n", (num1 + num2));

      b = fork();
      if (b == 0) {
         printf("\nchild1's first child ID is : %d\t Parent is %d\n", getpid(), getppid());
         printf(" Subtraction is %d\n", (num1 - num2));
      } else {
         int d = fork();
         if (d == 0) {
            printf("\nchild1's second child ID is : %d\t Parent is %d\n", getpid(), getppid());
            printf(" Multiplication is %d\n", (num1 * num2));
         } else {
            waitpid(b, &wstatus, 0);
         }
```

```
        }
    } else {
        c = fork();
        if (c == 0) {
            printf("\nParent Child2 ID is : %d\t Parent is %d\n", getpid(), getppid());
            printf(" Division is %d\n", (num1 / num2));
        } else {
            waitpid(a, &wstatus, 0);
        }
    }

    return (EXIT_SUCCESS);
}
```

3. Display “Hello World” message by 3 different threads

```
#include <stdio.h>
#include <stdlib.h>
void* fun(void *arg)
{

   printf("Hello World\n");



   pthread_exit(NULL);
}
int main(int argc, char** argv) {
 pthread_t t_id[3];

   int i=0;
   for(i=0;i<3;i++)
   {
   int x=pthread_create(&t_id[i],NULL,fun,NULL);
    if(x==0)
   {
      printf("Thread %d created successfully\n",(i+1));

   }
   else
   {
        printf("Failed to created thread\n");
   }
    printf("Thread id is %d\n",t_id[i]);
   }
   return (EXIT_SUCCESS);
}
```

4. Create two threads Thread1 adds marks (out of 10) of student1 from subject1 to subject5, Thread2 adds marks of student2 from subject1 to subject 5. Main process takes the sum returned from the Thread1 and Thread2, decides who scored more marks and displays student with its highest score.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

void* fun(void *arg) {
   int *marks = (int*)arg;
   int *sum = (int*)malloc(sizeof(int));
   *sum = 0;

   for (int i = 0; i < 5; i++) {
      *sum += marks[i];
   }

   return (void*)sum;
}

int main() {
   int stu1[5] = {50, 60, 39, 69, 70};
   int stu2[5] = {80, 66, 59, 99, 60};
   int *result1, *result2;
   pthread_t tid1, tid2;

   pthread_create(&tid1, NULL, fun, stu1);
   pthread_create(&tid2, NULL, fun, stu2);

   pthread_join(tid1, (void**)&result1);
```

```
    pthread_join(tid2, (void**)&result2);

    if (*result1 > *result2) {
        printf("stu1 has highest marks: %d\n", *result1);
    } else if (*result2 > *result1) {
        printf("stu2 has highest marks: %d\n", *result2);
    } else {
        printf("Both students have the same marks: %d\n", *result1);
    }

    free(result1);
    free(result2);

    return 0;
}
```

5. Implement the Dining Philosopher problem using POSIX threads

```
#include <stdio.h>
#include <semaphore.h>
#include <pthread.h>
#include <unistd.h>

#define N 5         // Number of philosophers
#define HUNGRY 1
#define THINKING 2
#define EATING 0
#define LEFT (phnum + N - 1) % N // Left philosopher
#define RIGHT (phnum + 1) % N    // Right philosopher

int state[N];       // State of philosophers
int phil[N] = {0,1,2,3,4};         // Philosopher IDs

sem_t mutex;         // Semaphore to protect critical sections
sem_t s[N];          // Semaphore for each philosopher

void test(int phnum) {
   if (state[phnum] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
      state[phnum] = EATING;
      printf("Philosopher %d takes forks %d and %d\n", phnum + 1, LEFT + 1, RIGHT + 1);
      printf("Philosopher %d is eating\n", phnum + 1);
      sleep(1); // Simulate eating
      sem_post(&s[phnum]); // Allow philosopher to proceed
   }
}

void take_fork(int phnum) {
   sem_wait(&mutex);        // Enter critical section
   state[phnum] = HUNGRY;
   printf("Philosopher %d is Hungry\n", phnum + 1);
```

```
    test(phnum);            // Try to acquire forks
    sem_post(&mutex);       // Exit critical section
    sem_wait(&s[phnum]);    // Wait until forks are available
}

void put_fork(int phnum) {
    sem_wait(&mutex);       // Enter critical section
    state[phnum] = THINKING;
    printf("Philosopher %d puts down forks %d and %d\n", phnum + 1, LEFT + 1, RIGHT + 1);
    printf("Philosopher %d is thinking\n", phnum + 1);
    test(LEFT);             // Notify left philosopher
    test(RIGHT);            // Notify right philosopher
    sem_post(&mutex);       // Exit critical section
}

void *philosopher(void *num) {
    int *phnum = (int *)num;
    sleep(1);           // Thinking
    take_fork(*phnum);  // Try to eat
    sleep(1);           // Eating
    put_fork(*phnum);   // Release forks
    return NULL;
}

int main() {
    pthread_t thread_id[N];

    sem_init(&mutex, 0, 1);  // Initialize mutex semaphore
    for (int i = 0; i < N; i++) {
        sem_init(&s[i], 0, 0); // Initialize semaphores for each philosopher
        phil[i] = i;         // Set philosopher IDs
        state[i] = THINKING;  // Initialize states
    }
```

```
    // Create philosopher threads
    for (int i = 0; i < N; i++) {
        pthread_create(&thread_id[i], NULL, philosopher, &phil[i]);
    }

    // Wait for all threads to finish
    for (int i = 0; i < N; i++) {
        pthread_join(thread_id[i], NULL);
    }

    return 0;
}
```

6. Implement Priority Scheduling Algorithm.

```
//Priority Scheduling Algorithm

#include <stdio.h>
#include <stdlib.h>

int main(int argc, char** argv) {
int i,j,N,TAT[10],WT[10],pos,temp;
 int Sum_wt=0,Sum_tat=0,AT[10],BT[10],PR[10];
 printf("Enter Total Process:\t ");
 scanf("%d",&N);

 for(i=0;i<N;i++)
 {
  printf("Enter Arrival Time for Process  %d :",i+1);
  scanf("%d",&AT[i]);
  printf("Enter Burst Time for Process  %d :",i+1);
  scanf("%d",&BT[i]);
  printf("Enter Priority of Process  %d :",i+1);
  scanf("%d",&PR[i]);
 }
 //Sort the processes according to priority
  for(i=0;i<N;i++)
 {
    pos=i;
    for(j=i+1;j<N;j++)
    {
       if(PR[j]<PR[pos])
       {
          pos=j;
       }
```

```
    }
    temp=PR[i];
    PR[i]=PR[pos];
    PR[pos]=temp;
    temp=BT[i];
    BT[i]=BT[pos];
    BT[pos]=temp;
  }
  //To find turnaround time and waiting time
  WT[0]=0;
  printf("\nProcess\t AT\t BT \t PR\t TAT\tWT\n");
  for(i=0;i<N;i++)
  {
    WT[i]=0;
    TAT[i]=0;
    for(j=0;j<i;j++)
    {
       WT[i]=WT[i]+BT[j];
    }
    TAT[i]=WT[i]+BT[i];
    Sum_wt=Sum_wt+WT[i];
    Sum_tat=Sum_tat+TAT[i];
     printf("P[%d]\t%d\t%d\t%d\t%d\t%d\n",i+1,AT[i],BT[i],PR[i],TAT[i],WT[i]);
  }
   printf("\nAverage Waiting Time= %f\n",(Sum_wt*1.0)/N);
  printf("Avg Turnaround Time = %f\n\n",(Sum_tat*1.0)/N);
   return (EXIT_SUCCESS);
}
```

7. Implement Round Robin Scheduling Algorithm.

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char** argv) {
int i,j,N,time,remain,count=0,TQ,TAT,WT;
 int Sum_wt=0, Sum_tat=0, AT[10], BT[10], Temp[10];
 printf("Enter Total Process:\t ");
 scanf("%d",&N);
 remain=N;
 for(i=0;i<N;i++)
 {
  printf("Enter Arrival Time for Process  %d :",i+1);
  scanf("%d",&AT[i]);
  printf("Enter Burst Time for Process  %d :",i+1);
  scanf("%d",&BT[i]);
  Temp[i]=BT[i];
 }
 printf("Enter Time Quantum:\t");
 scanf("%d",&TQ);
 printf("\n\nProcess\t|AT | BT |Turnaround Time|Waiting Time\n\n");
 for(time=0,i=0;remain!=0;)
 {
  if(Temp[i]<=TQ && Temp[i]>0)
  {
   time=time+Temp[i];
   Temp[i]=0;
   count=1;
  }
  else if(Temp[i]>0)
  {
   Temp[i]=Temp[i]-TQ;
```

```
      time=time+TQ;
    }
    if(Temp[i]==0 && count==1)
    {
      remain--;
      TAT=time-AT[i];
      WT=TAT-BT[i];
      printf("P[%d]\t|%d\t%d\t\t%d\t|\t%d\n",i+1,AT[i],BT[i],TAT,WT);
      Sum_wt=Sum_wt+WT;
      Sum_tat=Sum_tat+TAT;
      count=0;
    }
    if(i==N-1)
      i=0;
    else if(AT[i+1]<=time)
      i++;
    else
      i=0;
}
  printf("\nAverage Waiting Time= %f\n",(Sum_wt*1.0)/N);
  printf("Avg Turnaround Time = %f\n\n",(Sum_tat*1.0)/N);

    return (EXIT_SUCCESS);
}
```